



UNIVERSITETET I BERGEN

KANDIDAT

151

PRØVE

INF122 0 Funksjonell programmering

Emnekode	INF122
Vurderingsform	Skriftlig eksamen
Starttid	20.02.2026 08:00
Sluttid	20.02.2026 11:00
Sensurfrist	--
PDF opprettet	22.05.2026 16:31

Info

Oppgave	Tittel	Oppgavetype
i	Velkommen	Informasjon eller ressurser

Oppgaver

Oppgave	Tittel	Oppgavetype
1	Verdier i Haskell	Flervalg
2	Typer og Kinds i Haskell	Flervalg
3	IO	Programmering
4	Listebehandling	Programmering
5	Map og egendefinerte datastrukturer	Programmering
6	Induktive datastrukturer	Programmering

1 Verdier i Haskell

Hva er verdien av `head (tail [1,2,3])` ?

Velg ett alternativ:

- ☐ [1]
- ☒ 2
- ☐ [2,3]
- ☐ 1
- ☐ Ingen verdi, grunnet kjørefeil/error

Hva er verdien av `snd ([1,2],3)` ?

Velg ett alternativ

- ☒ 3
- ☐ (2,3)
- ☐ [2,3]
- ☐ 2
- ☐ Ingen verdi, grunnet typefeil

Hva er verdien av `length [2,4,6,8] / 2` ?

Velg ett alternativ

- ☒ 2
- ☐ Ingen verdi, grunnet kjørefeil/error
- ☐ [1,2,3,4]
- ☐ 4
- ☐ Ingen verdi, grunnet typefeil

Hva er verdien av `length [1,4..13]` ?

Velg ett alternativ

- ☐ 4
- ☐ 3
- ☒ Ingen verdi, grunnet kjørefeil/error

- ☐ 13
- ☐ Ingen verdi, grunnet typefeil
- ☐ 5

Hva er verdien av `length [1, 4..5]`?

Velg ett alternativ

- ☒ Ingen verdi, grunnet typefeil
- ☐ 2
- ☐ 3
- ☐ Ingen verdi, grunnet kjørefeil/error

Hva er verdien av `map sort ["en", "to", "tre"]` (Anta at vi har importert `sort` fra `Data.List`)?

Velg ett alternativ

- ☐ ["tre", "to", "en"]
- ☐ ["en", "to", "tre"]
- ☐ Ingen verdi, grunnet kjørefeil/error
- ☒ ["en", "ot", "ert"]
- ☐ ["en", "ert", "ot"]
- ☐ Ingen verdi, grunnet typefeil

Maks poeng: 12

2 Typer og Kinds i Haskell

Hva er en gyldig typing av `uncurry (+)`?

Velg ett alternativ:

- ☐ `Integer -> (Integer, Integer)`
- ☐ `(Integer -> Integer) -> Integer`
- ☐ `(Integer , Integer) -> Integer`
- ☒ `Integer -> (Integer -> Integer)`
- ☐ Uttrykket gir en typefeil.

Hvilket av alternativene er **ikke** en gyldig typing av 3?

Velg ett alternativ

- ☐ `Integer`
- ☐ `Num a => a`
- ☒ `Float`
- ☐ `Eq a => a`

Hva er en gyldig typing av `(:)`?

Velg ett alternativ

- ☐ `[a -> a] -> a`
- ☐ `[a]`
- ☒ `a -> [a] -> [a]`
- ☐ `[a] -> [a -> a]`
- ☐ `[a -> a]`

Hva er en gyldig typing av `(.)`?

Husk at `f . g = \x -> f (g x)`.

Velg ett alternativ

- ☐ $(b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow c$
- ☐ $(a \rightarrow b) \rightarrow c \rightarrow b \rightarrow a \rightarrow c$
- ☒ $(a \rightarrow b) \rightarrow (b \rightarrow c) \rightarrow a \rightarrow c$
- ☐ $(a \rightarrow b) \rightarrow (c \rightarrow b) \rightarrow a \rightarrow c$

Hva er en gyldig typing av `map . (:)`?

Velg ett alternativ

- ☐ $(a \rightarrow a) \rightarrow [a \rightarrow a]$
- ☒ $a \rightarrow [a] \rightarrow [a]$
- ☐ Uttrykket gir en typefeil.
- ☐ $[a \rightarrow a] \rightarrow a \rightarrow [a]$
- ☐ $a \rightarrow [[a]] \rightarrow [[a]]$

Hvilken kind har `Map` fra `Data.Map`?

Velg ett alternativ

- ☒ $(* \rightarrow *) \rightarrow *$
- ☐ $* \rightarrow *$
- ☐ $* \rightarrow * \rightarrow *$
- ☐ $*$

Maks poeng: 12

3 IO

I denne oppgaven skal du vise at du behersker grunnleggende IO programmering med `do`-notasjon i Haskell.

IO funksjonen `getChar :: IO Char` henter en enkelt `Char` som input fra brukeren.

a) [6 poeng]

Skriv en funksjon `getSentence :: IO String` som, ved hjelp av `getChar`, leser én og én `Char` helt til brukeren skriver inn ' . ', og deretter returnerer hele setningen (inklusive '.').

b) [8 poeng]

Skriv en `main :: IO ()` funksjon som leser inn en setning (ved hjelp av `getSentence` fra forrige oppgave), og skriver ut setningen med reversert ordrekkefølge. Ta høyde for å endre store/små bokstaver med `toUpper :: Char -> Char` og `toLowerCase :: Char -> Char`: Første ord i setningen skal ha stor forbokstav, og siste ord skal ikke ha det (se eksempel). Du kan anta at brukeren skriver inn en setning med minst ett ord, og at første ord i setningen ikke er et særnavn.

Enter sentence: Dette er en test.

Resultat: Test en er dette.

Skriv svaret ditt her. Endringer blir lagret automatisk.

```
1
2  getSentence :: IO
3  getSentence = do
4      let
5          char = getChar
6          word = word : char
7          if input != ' .' then getSentence else word
8
9  main :: IO()
10 main = do
11     print("Enter sentence: ")
12     let
13         sentence = getSentence
14         reversedSentence = map toLower (reverse sentence)
15         firstChar = toUpper (head (head (words reversedSentence)))
16         resultat = firstChar : tail reversedSentence
17     in
18     print("Resultat: " ++ resultat)
19
20
21
22
```

Maks poeng: 14

4 Listebehandling

denne oppgaven skal du vise at du behersker listebehandling, funksjonskomposisjon og høyereordens funksjoner i Haskell.

Her er noen nyttige funksjoner på lister:

```
map :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
concat :: [[a]] -> [a]
take :: Int -> [a] -> [a]
replicate :: Int -> a -> [a]
group :: (Eq a) => [a] -> [[a]]
```

a) [5 poeng]

Bruk listekomprehensjon til å skrive en funksjon `squareEvens :: [Integer] -> [Integer]` som returnerer kvadratet av alle partall i en liste. F.eks. skal `squareEvens [1,2,3,4]` evaluere til `[4,16]`.

b) [5 poeng]

Skriv samme funksjon som i a), men bruk `filter` og `map` istedet for listekomprehensjon.

c) [5 poeng]

Skriv en funksjon `countNothing :: [Maybe a] -> Integer` som teller antall forekomster av `Nothing` i en liste. For eksempel skal `countNothing [Nothing, Just "foo", Nothing, Just "bar"]` evaluere til 2. Merk at du **ikke** kan anta begrensningen `Eq a` eller `Eq (Maybe a)` i denne oppgaven.

d) [5 poeng]

Vi kan definere en funksjon:

```
dedupe :: (Eq a) => [a] -> [a]
dedupe = concat . map (take 1) . group
```

Gi en alternativ definisjon av `dedupe` som bruker mønstermatching og rekursjon i stedet for `concat`, `map` og `group`.

e) [5 poeng]

La `dupe = concat . map (replicate 2)`. Er `dedupe (dupe x) = x` for enhver liste `x :: [Integer]`? Husk å begrunne svaret ditt presist.

Skriv svaret ditt her. Endringer blir lagret automatisk.

```
1 squareEvens :: [Integer] -> [Integer]
2 squareEvens xs = [x^2 | x <- xs, even x] -- I don't remember if exponent is ^, ** or
3
4 squareEvens :: [Integer] -> [Integer]
5 squareEvens xs = filter even (map (^2 xs))
6
7 countNothing :: [Maybe a] -> Integer
8 countNothing xs = length (filter Nothing xs)
9
10 dedupe :: (Eq a) => [a] -> [a]
11 dedupe = concat . map (take 1) . group
12
13 dedupe :: (Eq a) => [a] -> [a]
14 dedupe [] = []
```



```
15 dedupe (x:xs) = x : [dedupe xs]
16
17 e)
18
19 dupe = concat . map (replicate 2)
20 dedupe = concat . map (take 1) . group
21
22
23 dupe [1, 2, 3..] = [11, 22, 33..]
24 -- iterates through the list and replicats the element then concatenates the result
25
26 -- iterates through the list, takes the first element and groups it and concat into
27 dedupe [11, 22, 33..] =[1, 2, 3..]
28 -- thus dedupe (dupe x) = x.
```

Maks poeng: 25

5 Map og egendefinerte datastrukturer

I denne oppgaven skal du vise at du forstår `Maybe` og `Data.Map`. Vi skal implementere en liten modell av et inventar- og kassesystem for en butikk.

Typene vi skal bruke i modellen er de følgende:

```
module Shop where
import Data.Map (Map)
import qualified Data.Map as Map
type Money = Integer
type Amount = Integer
type ItemID = Integer
data ItemInfo = ItemInfo { name :: String
                          , price :: Money }
    deriving (Eq, Show)
type ProductBase = Map ItemID ItemInfo
type Inventory = Map ItemID Amount
data Offer = Percent Integer
           | ThreeForTwo
    deriving (Eq, Show)
data BillItem = Purchase ItemID ItemInfo
              | Discount Money
    deriving (Eq, Show)
type Bill = [BillItem]
data Shop = Shop { inventory :: Inventory
                  , products  :: ProductBase
                  , offers    :: Map ItemID Offer
                  , till      :: Money }
```

Som du kan se, har butikken et inventar av varer, en database over alle produkter, og en oversikt over alle gjeldende tilbud. En `ItemID` identifiserer en type av produkter. For eksempel, kan en enkel butikk beskrives som dette:

```
-- Initial shop setup
initialShop :: Shop
initialShop = Shop
  { inventory = Map.fromList
    [ (1, 100), (2, 50), (3, 75), (4, 30), (5, 60) ]
  , products = Map.fromList
    [ (1, ItemInfo "Apple" 10)
    , (2, ItemInfo "Banana" 15)
    , (3, ItemInfo "Orange" 12)
    , (4, ItemInfo "Grape" 25)
    , (5, ItemInfo "Mango" 30)
    ]
  , offers = Map.fromList
    [ (1, Percent 10)  -- 10% off apples
    , (2, ThreeForTwo) -- 3-for-2 on bananas
    , (4, Percent 20)  -- 20% off grapes
    ]
  , till = 1000 -- Starting cash
  }
```

Her kan vi, for eksempel, se at butikken har 50 bananer i inventaret og at hver banan selges for 15kr, med 3-for-2 avslag. For eksempel, dersom man kjøper 5 bananer betaler får man avslag for én banan: $5 \cdot 15\text{kr} - 15\text{kr} = 60\text{kr}$.

a) [6 poeng]

Skriv en funksjon som regner ut hvor mye avslag man får dersom man får et prosentavslag på en pris, rundet av til nærmeste hele krone. For eksempel, dersom man får 30 prosent avslag på en vare som koster 201kr får man et avslag på 60kr. Altså må `computePercentDiscount 30 201 = 60`. Gjør mellomberegninger i `Double`.

```
computePercentDiscount :: Integer -> Money -> Money
```

Hint: Husk at vi har `round :: (RealFrac a, Integral b) => a -> b` og `fromInteger :: Num a => Integer -> a`.

b) [6 poeng]

Skriv en funksjon `takeItem` som tar ut et element fra inventaret. Funksjonen skal kun fjerne et item fra inventaret dersom det er minst ett item der av denne typen. Dersom Amount blir 0 etter at elementet er tatt ut, skal ItemID-nøkkelen fjernes fra inventarets Map.

```
takeItem :: ItemID -> Inventory -> Inventory
```

Du kan bruke `Map.update`, som kanskje oppdaterer en verdi i et map. Her er Haskell-dokumentasjonen for `update`:

```
update :: Ord k => (a -> Maybe a) -> k -> Map k a -> Map k a
```

`O(log n)`. The expression `(update f k map)` updates the value `x` at `k` (if it is in the map). If `(f x)` is `Nothing`, the element is deleted. If it is `(Just y)`, the key `k` is bound to the new value `y`.

```
let f x = if x == "a" then Just "new a" else Nothing
update f 5 (fromList [(5,"a"), (3,"b")]) == fromList [(3, "b"), (5, "new a")]
update f 7 (fromList [(5,"a"), (3,"b")]) == fromList [(3, "b"), (5, "a")]
update f 3 (fromList [(5,"a"), (3,"b")]) == singleton 5 "a"
```

c) [8 poeng]

En `Bill` er en liste med varer som kjøpes og eventuelle avslag som skal regnes inn i totalprisen. Når en ny vare scannes i kassen legges den og alle gyldige avslag til på `Bill`-listen. Vi skal nå skrive `addItem` som legger til et slikt element. Vi begynner med å slå opp `ItemInfo` om produktet i `shop :: Shop`, og lagre det i `info`. Din oppgave er å regne ut eventuelle avslag på prisen, som en `Maybe` verdi, ved å se i butikkens tilbudsoversikt `offers shop :: Map ItemID Offer`. Skriv inneholdet i `discount :: Maybe BillItem`, nedenfor:

```
consMaybe :: Maybe a -> [a] -> [a]
consMaybe Nothing as = as
consMaybe (Just a) as = a:as
addItem :: Shop -> ItemID -> Bill -> Bill
addItem shop item bill = consMaybe discount $ consMaybe purchase bill where
    info :: Maybe ItemInfo
    info = Map.lookup item (products shop)
    purchase :: Maybe BillItem
    purchase = Purchase item <$> info
    discount :: Maybe BillItem
    discount = {-TODO: Regn ut et eventuelt prisavslag her-}
```

Hint: For å finne ut om det skal være 3-for-2 rabatt, så må du se i `bill` om antall elementer av denne typen etter det gjeldende elementet er lagt til blir delelig på 3.

Husk at du kan bruke `Map.lookup :: Ord k => k -> Map k a -> Maybe a` for å slå opp en verdi i et `Data.Map`.

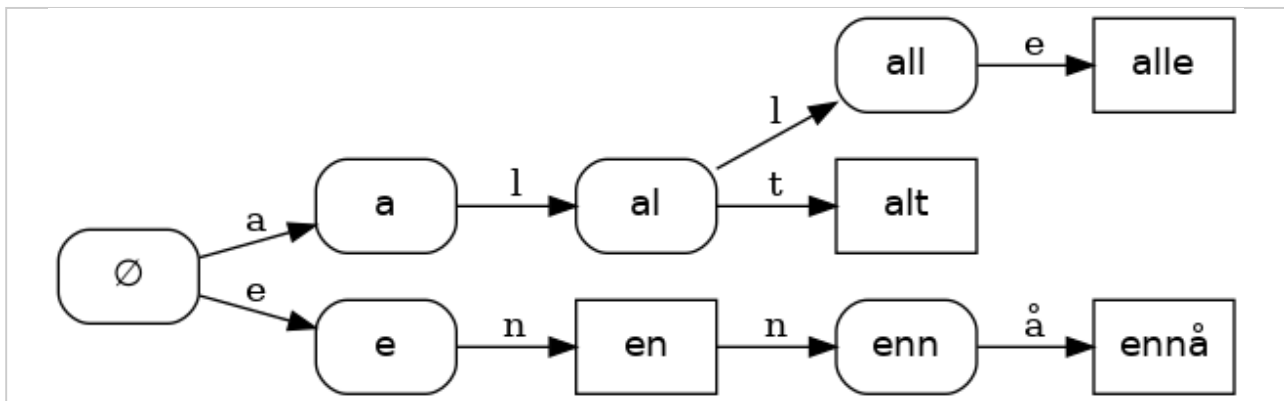
Skriv svaret ditt her. Endringer blir lagret automatisk.

1	<code>computePercentDiscount :: Integer -> Money -> Money</code>
2	<code>computePercentDiscount percent value = let</code>
3	<code>..... factor = 1 - (percent / 100)</code>
4	<code>..... in round (value * factor)</code>
5	
6	
7	
8	

```
9 takeItem :: ItemID -> Inventory -> Inventory
10 takeItem id inven = update id -1 inven
11
12
13 consMaybe :: Maybe a -> [a] -> [a]
14 consMaybe Nothing as = as
15 consMaybe (Just a) as = a:as
16 addItem :: Shop -> ItemID -> Bill -> Bill
17 addItem shop item bill = consMaybe discount $ consMaybe purchase bill where
18     info :: Maybe ItemInfo
19     info = Map.lookup item (products shop)
20     purchase :: Maybe BillItem
21     purchase = Purchase item <$> info
22     discount :: Maybe BillItem
23     discount =
24         if (Map.lookup item offers) != Nothing then
25             if length filter item bill `mod` 3 == 0 then
26                 purchase item <$> money - (money / 3)
27             else purchase item <$> money * (computePercentDiscount Map.lookup offers item)
```

Maks poeng: 20

6 Induktive datastrukturer



I denne oppgaven skal du vise at du forstår induktive datastrukturer.

Vi skal se på trær som representerer mengder av ord. For eksempel representerer treet ovenfor mengden {"alt", "alle", "en", "ennå"}. Avrundede noder er uavsluttede ord, og rektangler er avsluttede.

Vi skal bruke følgende induktive data struktur for å representere treet:

```
data WordTree = WordTree { endAllowed :: Bool
                           , continuations :: [(Char, WordTree)] }
    deriving (Eq, Show, Ord)
```

Feltet `endAllowed :: WordTree -> Bool` representerer hvorvidt det er mulig å avslutte ordet på dette punktet.

Feltet `continuations :: WordTree -> [(Char, WordTree)]` representerer alle gyldige fortsettelser av treet. Vi skal ha som invariant at dette er en assosiasjonsliste, altså at en `Char`-verdi kan kun forekomme i ett tuppel i listen. Denne listen assosierer dermed hver `Char` som kan forekomme til et tre som representerer alle videre fortsettelser.

Husk at man kan slå opp i assosiasjonslister ved hjelp av:

```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

Her er treet ovenfor skrevet ut i Haskell:

```
myWordTree :: WordTree
myWordTree =
  WordTree False
    [ ('a'
      , WordTree False
        [ ('l'
          , WordTree False
            [ ('l'
              , WordTree False
                [ ('e', WordTree True [])
                ])
              , ('t', WordTree True [])
            ])
          , ('e'
            , WordTree False
              [ ('n'
                , WordTree True
                  [ ('n'
                    , WordTree False
                      [ ('å', WordTree True [])
                    ])
                ])
              ])
        ])
    ]
```

```

    ])
  ])
]

```

a) [7 poeng]

Skriv en funksjon `insertWord` som tar inn en streng og et tre, og lager et nytt tre som legger strengen som et nytt mulig ord i treet.

```
insertWord :: String -> WordTree -> WordTree
```

b) [4 poeng]

Forklar hvordan funksjonen du skrev i a) bevarer invarianten beskrevet ovenfor.

Du skal altså vise at dersom `continuations tree` er en assosiasjonsliste, så er også `continuations (insertWord w tree)` en assosiasjonsliste for en hver `w :: String`.

c) [6 poeng]

Skriv en funksjon som henter ut alle mulige ord fra et `WordTree`:

```
allWords :: WordTree -> [String]
```

For eksempel blir `allWords myWordTree == ["alle", "alt", "en", "ennå"]`. Rekkefølgen på elementene i listen som `allWords` returnerer er likegyldig.

Skriv svaret ditt her. Endringer blir lagret automatisk.

```

1  myWordTree :: WordTree
2  myWordTree =
3      WordTree False
4      [ ('a'
5          , WordTree False
6          [ ('l'
7              , WordTree False
8              [ ('l'
9                  , WordTree False
10                     [ ('e', WordTree True [])
11                         ])
12                     , ('t', WordTree True [])
13                     ])
14                 ])
15             , ('e'
16                 , WordTree False
17                 [ ('n'
18                     , WordTree True
19                     [ ('n'
20                         , WordTree False
21                         [ ('å', WordTree True [])
22                             ])
23                         ])
24                     ])
25             ])
26
27 insertWord :: String -> WordTree -> WordTree
28 insertWord [] _ = []
29 insertWord word WordTree = if length (tail word) != 1
30                             then wordTree : [(head word), WordTree False] : insertWd
31                             else wordTree : [(head word), WordTree True]
32
33 allWords :: WordTree -> [String]
34 allWords _ = []
35 allWords wordTree (End Continue) = let
36     char = Continue (char _)
37     word = word ++ char
38     list = []
39     if (End != True)

```

Maks poeng: 17